

Memoria práctica de minero con hilos y procesos



-DANIEL GONZÁLEZ URETA
-ANTONINO ALBARRÁN PEÑAS

6 de marzo de 2025

1. Requisitos cumplidos y no cumplidos

Requisitos que se han cumplido:

En esta primera práctica hemos implementado un programa de tipo miner. Para esto creamos dos procesos mediante la función `fork()`:

- El proceso padre: Que será el proceso minero.
- El proceso hijo: Un proceso registrador, el logger.

Antes de crear los dos procesos mediante el `fork()`, se ha creado un pipe para que ambos procesos puedan comunicarse. Mientras que el proceso padre utilizará `fd[1]` donde escribirá, el registrador utilizará `fd[0]` donde leerá los resultados y los guardará en un fichero `log`.

Proceso Minero:

El proceso padre ejecutará varias rondas, tantas como el usuario le indique por terminal al ejecutar el programa (`n_rounds`). Dentro de cada ronda creará varios hilos utilizando la función `pthread_create` para dividir el trabajo de búsqueda en bloques de tamaño (`POW_LIMIT/ n_threads`), siendo también el usuario el responsable de indicar por terminal cuantos hilos quiere que se ejecuten. Cada hilo utilizará la función `pow_hash` para intentar encontrar la solución, y una vez que uno de los hilos la encuentren los demás se detendrán.

Proceso Registrador:

Se encarga de leer los resultados desde la tubería y generará un fichero `log` donde escribirá la información obtenida en cada ronda.

Problemas en el código:

El uso de variables globales, que no es nada recomendable, no teníamos ninguna otra opción en esta práctica con los conocimientos que tenemos de plantearlo de otra forma.

2.Pruebas Realizadas:

Un ejemplo de ejecución del programa es el siguiente, donde 1000 es el target, 5 es el número de rondas y 4 el número de hilos:

```
dani@dani-730QFG:~/Documentos/SOPER$ gcc miner.c pow.c -o miner -lpthread
dani@dani-730QFG:~/Documentos/SOPER$ ./miner 1000 5 4
```

Y el resultado obtenido es el siguiente:

Obtenemos la siguiente salida por pantalla:

```
dani@dani-730QFG:~/Documentos/SOPER$ ./miner 1000 5 4
Solution accepted: 00001000 --> 07871333
Solution accepted: 07871333 --> 05783163
Solution accepted: 05783163 --> 09078042
Solution accepted: 09078042 --> 09677760
Solution accepted: 09677760 --> 05618253
Logger exited with status 0
Miner exited with status 0
dani@dani-730QFG:~/Documentos/SOPER$
```

Y además se generará un fichero de tipo miner_<PID>.log como el siguiente, donde vemos la información producida y los resultados.

```
miner_10857.log
1  Id: 0
2  Winner: 10857
3  Target: 1000
4  Solution: 07871333 (validated)
5  Votes: 0/0
6  Wallets: 10857:0
7  -----
8  Id: 1
9  Winner: 10857
10 Target: 7871333
11 Solution: 05783163 (validated)
12 Votes: 1/1
13 Wallets: 10857:1
14 -----
15 Id: 2
16 Winner: 10857
17 Target: 5783163
18 Solution: 09078042 (validated)
19 Votes: 2/2
20 Wallets: 10857:2
21 -----
22 Id: 3
23 Winner: 10857
24 Target: 9078042
25 Solution: 09677760 (validated)
26 Votes: 3/3
27 Wallets: 10857:3
28 -----
29 Id: 4
30 Winner: 10857
31 Target: 9677760
32 Solution: 05618253 (validated)
33 Votes: 4/4
34 Wallets: 10857:4
35 -----
36
```

Casos límites que se han probado:

1. Número de hilos= 1

```
Miner exited with status 0
dani@dani-730QFG:~/Documentos/SOPER$ ./miner 231312 5 1
Solution accepted: 00231312 --> 02370900
Solution accepted: 02370900 --> 06238972
Solution accepted: 06238972 --> 02010154
Solution accepted: 02010154 --> 06144063
Solution accepted: 06144063 --> 07772273
Logger exited with status 0
Miner exited with status 0
```

En este caso se crea un solo hilo. El programa funciona igual, pero ese único hilo tendrá que recorrer desde start=0 a end=POW_LIMIT él solo, por lo que la ejecución será más lenta que si usamos más hilos.

2. Número de rondas=0

```
Miner exited with status 0
dani@dani-730QFG:~/Documentos/SOPER$ ./miner 231312 0 23
Miner: no hay rondas que ejecutar
dani@dani-730QFG:~/Documentos/SOPER$
```

En este caso el programa dará un error de entrada porque al menos necesita una ronda para poder buscar la solución . No se crearán los hilos internamente.

3. Número de hilos negativo:

```
miner: no hay fondos que ejecutar
dani@dani-730QFG:~/Documentos/SOPER$ ./miner 231312 23 -23
Error: el numero de hilos debe ser mayor que 0
dani@dani-730QFG:~/Documentos/SOPER$
```

En este caso también nos dará un error de entrada de los argumentos porque el número de hilos que se tienen que crear para intentar encontrar la solución debe ser mayor que 1.

4. Número de hilos creados muy grande:

```
dani@dani-730QFG:~/Documentos/SOPER$ ./miner 231312 5 10000
Solution accepted: 00231312 --> 02370900
Solution accepted: 02370900 --> 06238972
Solution accepted: 06238972 --> 02010154
Solution accepted: 02010154 --> 06144063
Solution accepted: 06144063 --> 07772273
Logger exited with status 0
Miner exited with status 0
dani@dani-730QFG:~/Documentos/SOPER$
```

Aunque se encuentre la solución correctamente, esta ejecución tarda muchísimo tiempo. Se están creando muchos hilos, luego el sistema operativo tiene que trabajar bastante y aunque parezca que al crearse muchos hilos cada hilo tendría menos tamaño que recorrer y por eso sería más rápido, no es así, porque los hilos compiten por un puesto en la CPU y el sistema operativo tiene que controlarlos por eso tarda tanto.

3.Funcionalidades:

Las funcionalidades que tiene nuestro programa es la escritura automática de logs en ficheros, donde se añade toda la información. La siguiente captura es un ejemplo de un fichero log

```
miner_7416.log
1 Id: 0
2 Winner: 7416
3 Target: 231312
4 Solution: 02370900 (validated)
5 Votes: 0/0
6 Wallets: 7416:0
7 -----
8 Id: 1
9 Winner: 7416
10 Target: 2370900
11 Solution: 06238972 (validated)
12 Votes: 1/1
13 Wallets: 7416:1
14 -----
15 Id: 2
16 Winner: 7416
17 Target: 6238972
18 Solution: 02010154 (validated)
19 Votes: 2/2
20 Wallets: 7416:2
21 -----
22 Id: 3
23 Winner: 7416
24 Target: 2010154
25 Solution: 06144063 (validated)
26 Votes: 3/3
27 Wallets: 7416:3
28 -----
29 Id: 4
30 Winner: 7416
31 Target: 6144063
32 Solution: 07772273 (validated)
33 Votes: 4/4
34 Wallets: 7416:4
35 -----
```

También se creó un pipe para mejorar la comunicación entre procesos, permitiendo que el proceso padre el minero escriba y el proceso hijo lea y cree los ficheros.log.

Por último cabe recalcar el uso de variables globales. Utilizamos dos, una que sea una flag que se ponga a uno si la solución ya está encontrada, sino permanecerá en 0, y otra que sea la solución.

Creamos la struct para guardar la información que debe conocer cada hilo, desde donde empieza, hasta donde va y cuál es el target que tiene que buscar.

Hemos creado la función mine que será la que ejecutará cada hilo para intentar encontrar la solución pedida.

```
9
10 volatile int solucion_found = 0;
11 long solucion;
12 typedef struct
13 {
14     long start;
15     long end;
16     long target; /*Numero de hilos*/
17 } thread_data;
18
19 /*funcion que ejecuta cada hilo. Recorre su rango hasta encontrar la solucion o que otro hilo la haya encontrado*/
20 void *mine(void *arg)
21 {
22     thread_data *data = (thread_data *)arg;
23     int i;
24     for (i = data->start; i < data->end && !solucion_found; i++)
25     {
26         if (pow_hash(i) == data->target)
27         {
28             solucion = i;
29             solucion_found = 1;
30         }
31     }
32     pthread_exit(NULL); /*salimos del hilo*/
33 }
34
```

Luego a esta función mine la pasaremos por argumento cuando creemos los hilos con pthread_create.

```
    if (pthread_create(&threads[j], NULL, mine, &thread_data[j]) != 0)
    {
        perror("pthread_create");
        exit(EXIT_FAILURE);
    }
}
```

Para el caso de simular que la solución sea no validada, hemos pensado que estaría bien simular un ejemplo. Para ello hemos acordado que tras 15 rondas, la solución sea rechazada. Y posteriormente una vez que una solución es rechazada no se hace break sino que se continúa en el bucle y ya la siguiente solución si que será validada.

```
if (solucion != -1)
{
    if ((i + 1) % 15 == 0)
    {
        /*Cada 15 vees se rechaza*/
        validada = 0;
    }
    else
    {
        validada = 1;
    }
    if (validada)
    {
        printf("Solution accepted: %08d --> %08ld\n", target, solucion);
    }
    else
    {
        printf("Solution rejected: %08d --> %08ld\n", target, solucion);
    }

    /*Ahora que hemos encontrado la solucion lo escribimos en la tuberia para enviarselo al hijo*/
    write(fd[1], &target, sizeof(int));
    write(fd[1], &solucion, sizeof(long));
    write(fd[1], &validada, sizeof(int));

    target = solucion;
}
else
{
    // Si alguna ronda falla, paramos
    printf("Miner: No se encontró solución en la ronda %d\n", i);
}
}
```

De esta manera en el proceso registrador tenemos en cuenta esa variable “validada”.

```
{
    read(fd[0], &r_solucion, sizeof(long));
    read(fd[0], &r_validated, sizeof(int));

    fprintf(f, "Id: %d\n", round);
    fprintf(f, "Winner: %d\n", getppid());
    fprintf(f, "Target: %d\n", r_target);
    fprintf(f, "Solution: %08ld (%s)\n", r_solucion, r_validated ? "validated" : "rejected");
    fprintf(f, "Votes: %d/%d\n", round, round);
    fprintf(f, "Wallets: %d:%d\n", getppid(), round);
    fprintf(f, "-----\n");
    fflush(f);
    round++;
}
fclose(f);
close(fd[0]);
printf("Logger exited with status 0\n");
exit(EXIT_SUCCESS);
}
```

Un ejemplo de ejecución para que veamos bien claro lo que pasa es el siguiente:

```
• dani@dani-730QFG:~/Documentos/SOPER$ ./miner 231312 17 5
Solution accepted: 00231312 --> 02370900
Solution accepted: 02370900 --> 06238972
Solution accepted: 06238972 --> 02010154
Solution accepted: 02010154 --> 06144063
Solution accepted: 06144063 --> 07772273
Solution accepted: 07772273 --> 02946962
Solution accepted: 02946962 --> 04270356
Solution accepted: 04270356 --> 05076308
Solution accepted: 05076308 --> 09365734
Solution accepted: 09365734 --> 01389545
Solution accepted: 01389545 --> 03505792
Solution accepted: 03505792 --> 00237561
Solution accepted: 00237561 --> 08896325
Solution accepted: 08896325 --> 08675622
Solution rejected: 08675622 --> 05592219
Solution accepted: 05592219 --> 07342521
Solution accepted: 07342521 --> 03354865
Logger exited with status 0
Miner exited with status 0
• dani@dani-730QFG:~/Documentos/SOPER$
```

Aquí solicitamos por terminal que se hagan 17 rondas. Pues al llegar a la ronda 15 el programa determina que la solución no sea validada por la condición que le hemos impuesto de que cada 15 rondas tenía que rechazar. La ronda 15 no será validada pero la 16 si que lo será y se calculará el target con la solución anterior que no ha sido validada. En el fichero.log que se crea lo vemos así:

```
Votes: 5/5
Wallets: 5169:9
-----
Id: 10
Winner: 5169
Target: 1389545
Solution: 03505792 (validated)
Votes: 10/10
Wallets: 5169:10
-----
Id: 11
Winner: 5169
Target: 3505792
Solution: 00237561 (validated)
Votes: 11/11
Wallets: 5169:11
-----
Id: 12
Winner: 5169
Target: 237561
Solution: 08896325 (validated)
Votes: 12/12
Wallets: 5169:12
-----
Id: 13
Winner: 5169
Target: 8896325
Solution: 08675622 (validated)
Votes: 13/13
Wallets: 5169:13
-----
Id: 14
Winner: 5169
Target: 8675622
Solution: 05592219 (rejected)
Votes: 14/14
Wallets: 5169:14
-----
Id: 15
Winner: 5169
Target: 5592219
Solution: 07342521 (validated)
Votes: 15/15
Wallets: 5169:15
-----
Id: 16
Winner: 5169
Target: 7342521
Solution: 03354865 (validated)
Votes: 16/16
Wallets: 5169:16
-----
```

Preguntas teóricas:

d) Pruebas y Razonamiento: Una vez programadas las diferentes partes, hacer pruebas con diferente número de rondas y diferente número de hilos, tomando tiempos de ejecución. ¿Qué conclusiones se pueden tomar respecto al número óptimo de hilos?

Para ver como varían los tiempos según el número de hilos que le introducimos por terminal lo mejor es hacer una simulación a gran escala con muchas rondas y donde vayamos aumentando uno a uno el número de hilos.

Para ellos podemos hacer un script que simula lo que queremos. En nuestro caso hemos pedido que el target sea 12345, que haga 800 rondas de búsqueda y que el número de hilos varía entre 1 y 32.

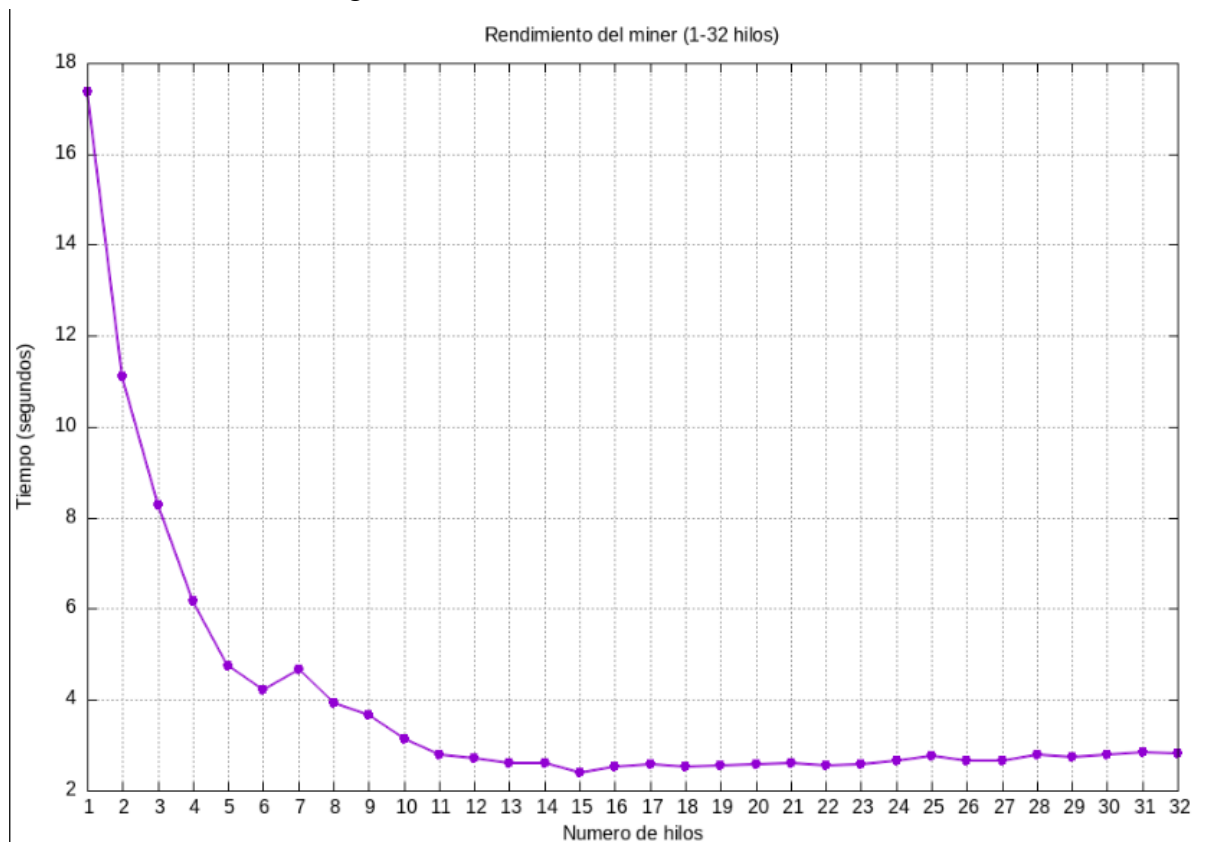
Este es el código del script que hemos creado.

```
$ pruebas.sh
1  #!/bin/bash
2
3  TARGET=12345
4  ROUNDS=800
5
6  echo "#Hilos Tiempo" > tiempos.dat
7
8  for H in $(seq 1 32)
9  do
10     echo "Probando $H hilos..."
11
12     TIME=$( (time ./miner $TARGET $ROUNDS $H) 2>&1 | grep real | awk '{print $2}' ) # extraemos solo el valor del tiempo real.
13
14     MIN=$(echo $TIME | cut -d'm' -f1)
15     SEC=$(echo $TIME | cut -d'm' -f2 | sed 's/s//' | sed 's/,./')
16
17     TOTAL=$(echo "$MIN*60 + $SEC" | bc -l) # convertimos el tiempo min/seg a segundos totales.
18
19     echo "$H $TOTAL" >> tiempos.dat
20 done
```

Tras ejecutarse se crea un fichero llamado tiempos.dat donde escribe el tiempo en segundos que ha tardado cada hilo. Es un fichero del siguiente tipo:

```
tiempos.dat
1  #Hilos Tiempo
2  1 17.375
3  2 11.099
4  3 8.287
5  4 6.170
6  5 4.748
7  6 4.220
8  7 4.657
9  8 3.938
10 9 3.660
11 10 3.123
12 11 2.792
13 12 2.716
14 13 2.598
15 14 2.619
16 15 2.408
17 16 2.531
18 17 2.570
19 18 2.529
20 19 2.557
21 20 2.588
22 21 2.590
23 22 2.557
24 23 2.584
25 24 2.650
26 25 2.755
27 26 2.659
28 27 2.666
29 28 2.798
30 29 2.744
31 30 2.790
32 31 2.848
33 32 2.815
```


Por último generamos una gráfica para ver mejor cómo va variando el tiempo dependiendo del número de hilos utilizados con gnuplot. La gráfica que hemos hecho, con los datos anteriores, nos sale de la siguiente manera:



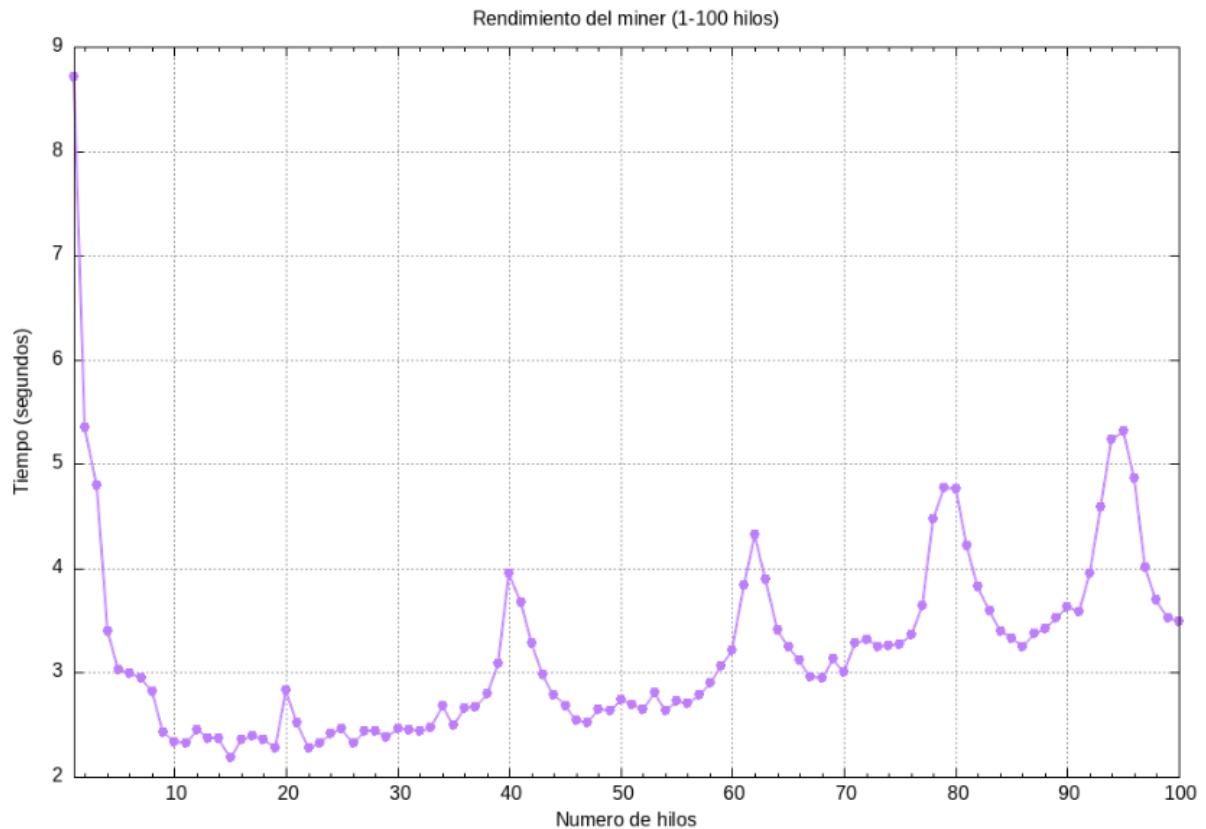
Como vemos en esta gráfica el tiempo menor es cuando estamos por 15-16 hilos. Eso coincide totalmente con lo que debería pasar teóricamente. En teoría nos han explicado que el tiempo mínimo de ejecución se obtiene aproximadamente cuando el número de hilos coincide con el número de núcleos del sistema. El ordenador donde hemos ejecutado ese script tiene 16 núcleos del sistema.

```
dani@dani-730QFG: ~  
dani@dani-730QFG:~$ nproc  
16
```

En conclusión podemos decir que el número óptimo va a ser 16, que coincide con el número de núcleos del sistema operativo que hemos utilizado para ejecutar esta práctica.

Para ver mejor cómo crece el tiempo una vez que superamos por mucho el número de hilos óptimos realizamos el mismo problema pero cambiando que itere desde 1 hilo hasta 100 hilos.

Obtenemos la siguiente gráfica:



Como vemos el menor tiempo está sobre los 15-16 hilos que es lo esperado. Una vez superado ese número de hilos, el tiempo empieza a crecer aunque haya algún que otro altibajo, por la concurrencia saturada del sistema operativo.